

Perceptrón simple y multicapa

TP3 · Fraude y dígitos manuscritos

Chiatellino, Máximo	63477
Coleur, Matías	63461
Curi Martínez, Gonzalo Sharif	63463
Novillo, Valentina	63156
Taurian, Magdalena	62828

Indice

- 01 Herramientas implementadas

- 02 Ejercicio 1: destilación de conocimiento (fraude)

- 03 Ejercicio 2: dígitos manuscritos

- 04 Ejercicio 3: más datos, meta de 98 %

- 05 Conclusiones

Implementación en C

Red en C11

Capas de neuronas, forward y backpropagation. El bias es `weights[0]` con entrada fija 1. Misma seed, mismo resultado.

Optimizador aparte

Backprop entrega el gradiente sin η y el optimizador decide el paso. No conoce neuronas, redes ni datasets.

Datos y entrenamiento

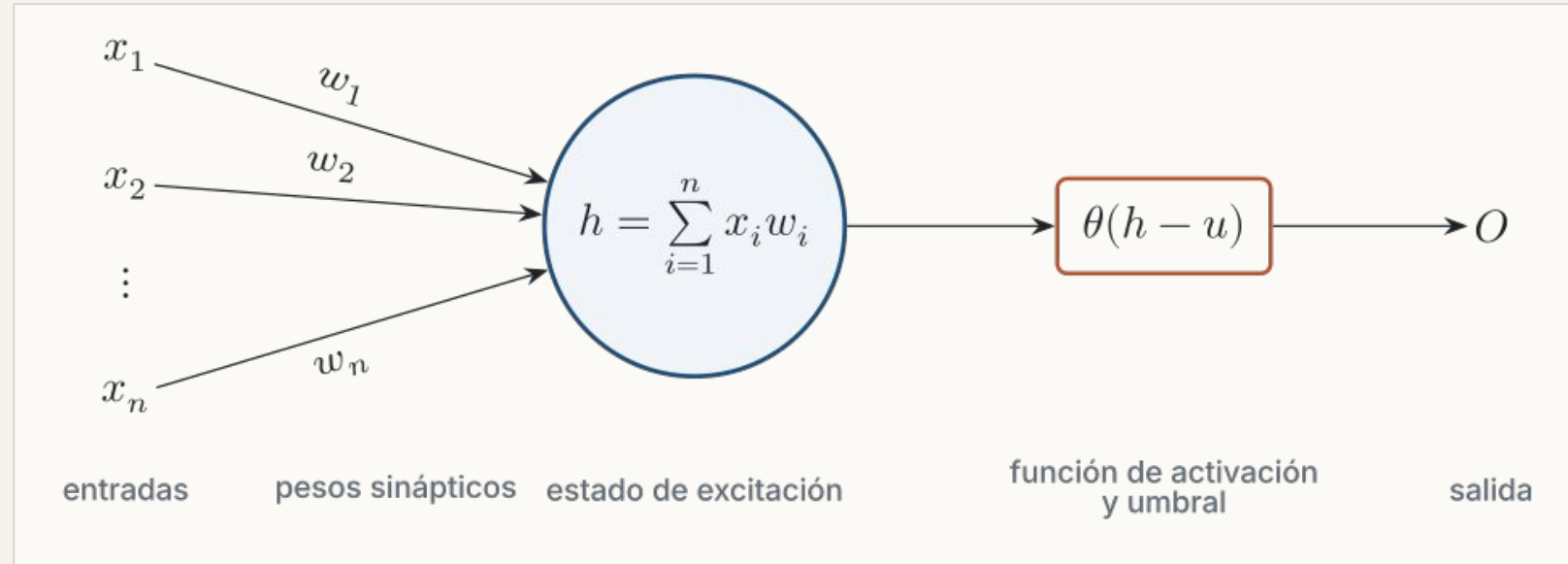
`batch_size` cubre online, mini-batch y batch. Shuffle por época y split de validación opcionales. Se conserva la mejor época de train.

Sin scikit-learn ni Keras: librerías externas solo para I/O, arrays y gráficos.

Implementación perceptrón

Struct neuron

- `int n_inputs`
- `double * weights`
- `double * gradient`
- `double * theta`
- `double * theta_prima`
- `optimizer`

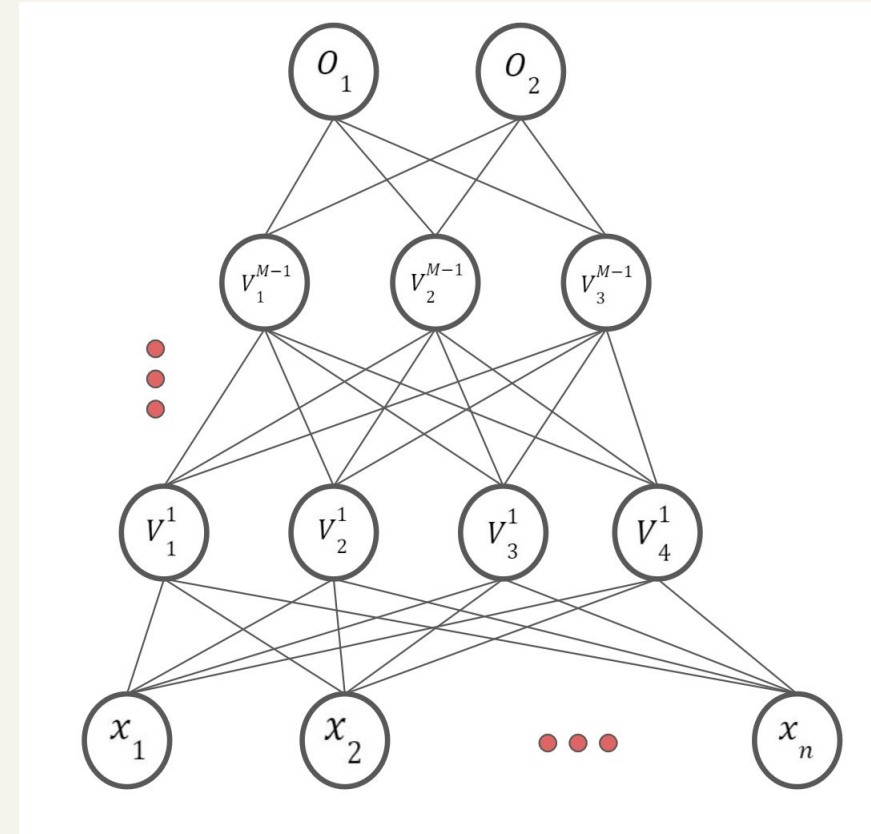


- Los pesos se inicializan con valores aleatorios entre -0.5 y 0.5
- $w[0] = bias$, inicializado en 1

Implementación network

Struct network

- *int* n_layers
- *int* * sizes
- *Neuron* ** layers
- *double* ** v
- *double* ** h
- *double* ** delta



Cuatro herramientas

sign

Escalón

Salida discreta. La regla delta con $\theta' = 1$ es exactamente la de Rosenblatt.

lineal

Lineal

Salida continua $\theta(h) = h$.
Descenso por gradiente sobre el error cuadrático.

tanh · logistic · relu

No lineal

Misma regla con θ' en el gradiente. Logistic cuando la salida es una probabilidad.

backprop

Multicapa

Capas ocultas y varias salidas. Sin capas ocultas es exactamente el perceptrón simple.

Optimizadores: GD, momentum, RMSProp, Adam y η adaptativo, en un módulo aparte.

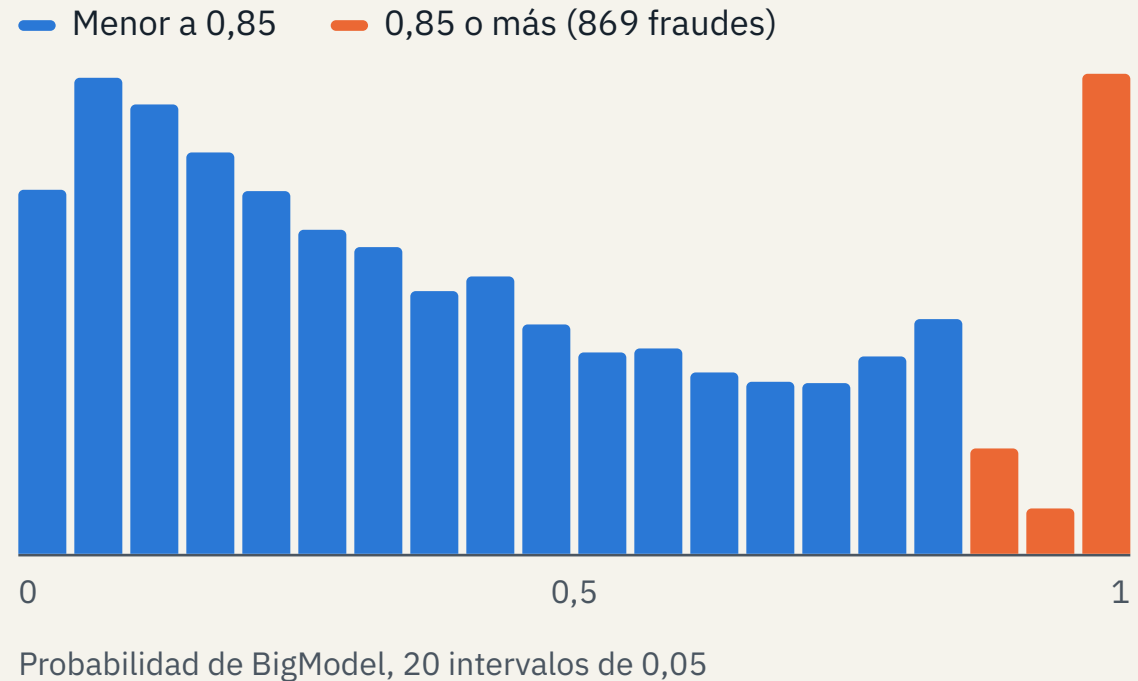
EJERCICIO 1

Knowledge distillation

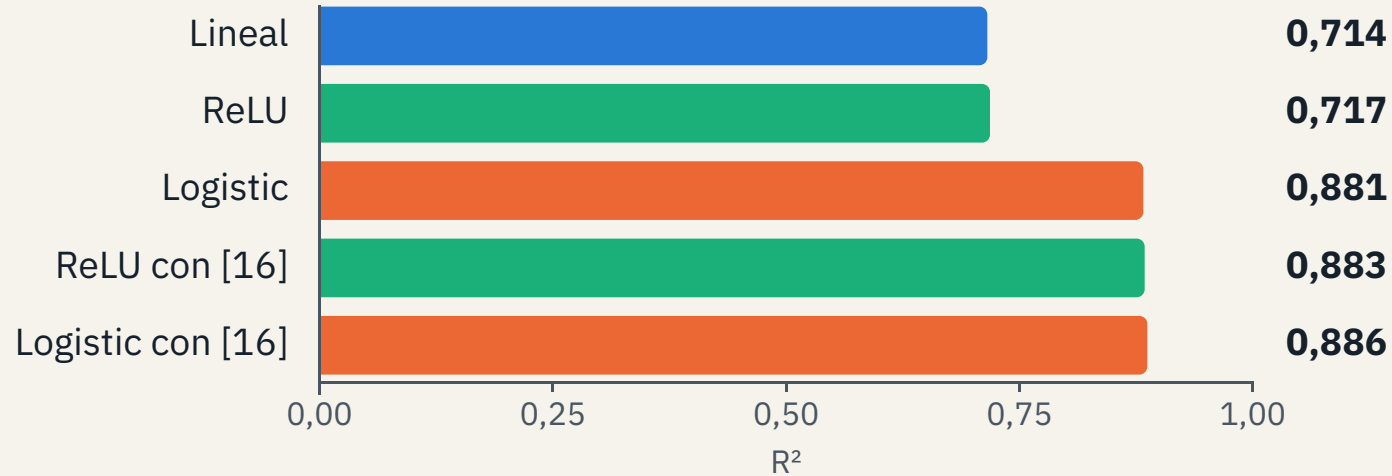
Un TinyModel que iguale a BigModel estimando la probabilidad de fraude.

Exploración del dataset

- 7500 transacciones, 9 entradas y la probabilidad de BigModel como objetivo
- Sin nulos, duplicados ni valores fuera de rango
- 11,6 % marcadas como fraude, todas con probabilidad de 0,85 o más
- timestamp, resolución de la pantalla y último login no se correlacionan con el objetivo
- flagged_fraud solo sirve para evaluar



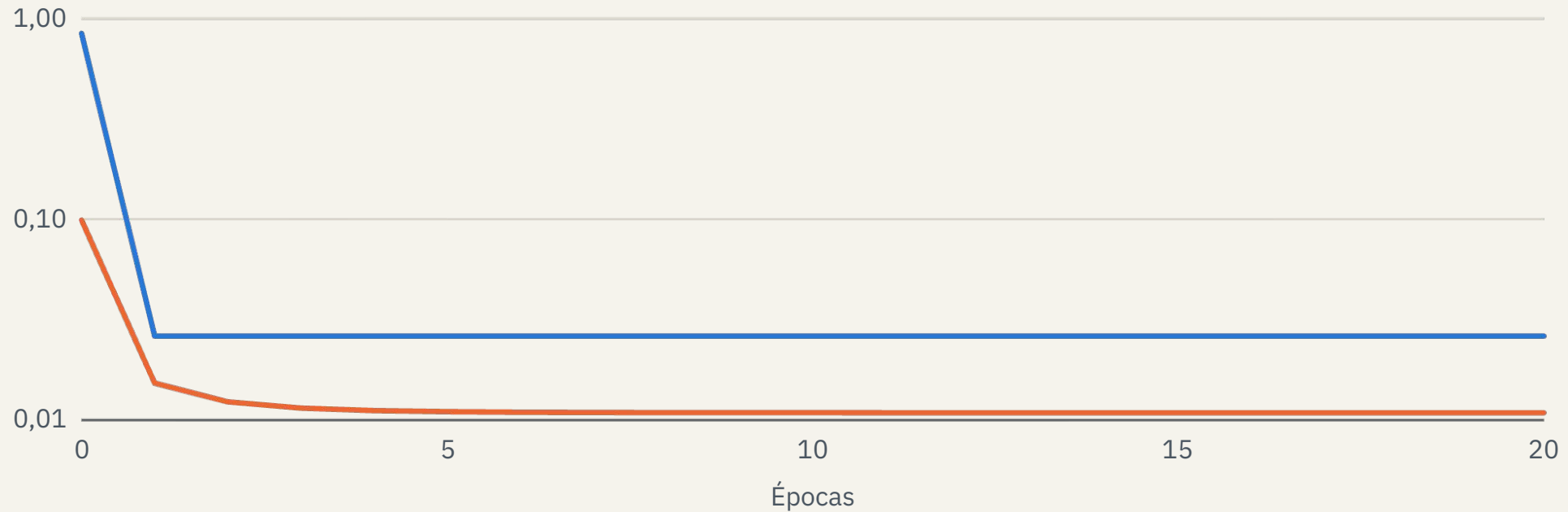
Lineal, logistic y ReLU



- El lineal tiene underfitting: explica 71 % de la varianza. ReLU sin capa oculta no mejora
- La logistic llega a 0,881, y una capa oculta suma solo 0,005: la capacidad ya no es el límite
- **Elegido: logistic.** ReLU con [16] la iguala, pero su salida no está acotada a (0, 1)

Curvas de aprendizaje

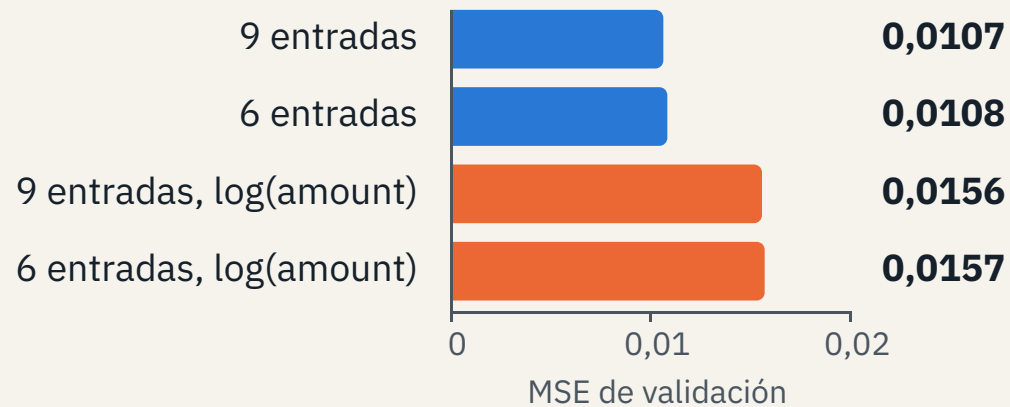
— Lineal (η 0,001) — Logistic (η 0,01)



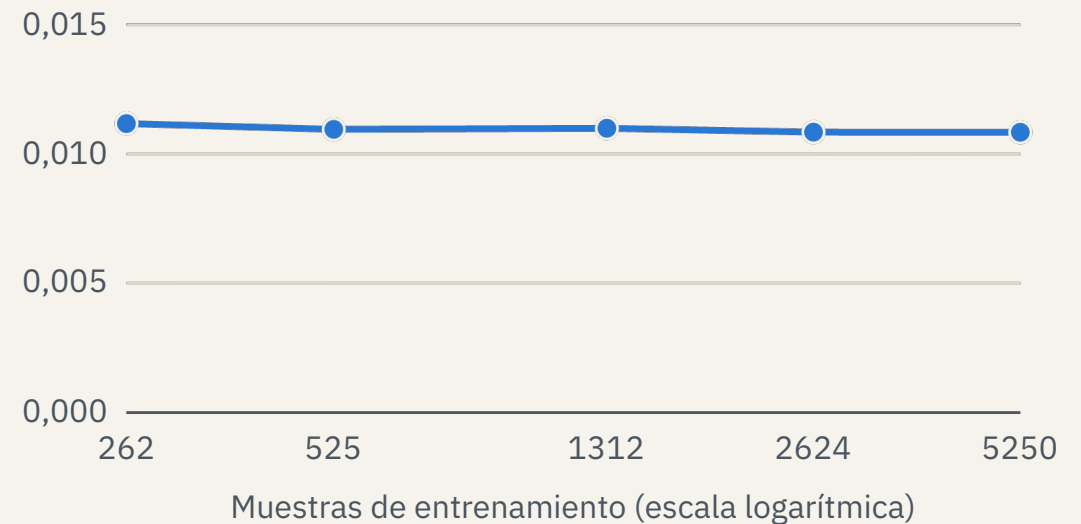
El lineal llega a su mínimo (MSE 0,026) en una época y no baja más. La logistic termina en 0,011.

Entradas y datos de entrenamiento

MSE de validación según las entradas



MSE de validación según el tamaño del train

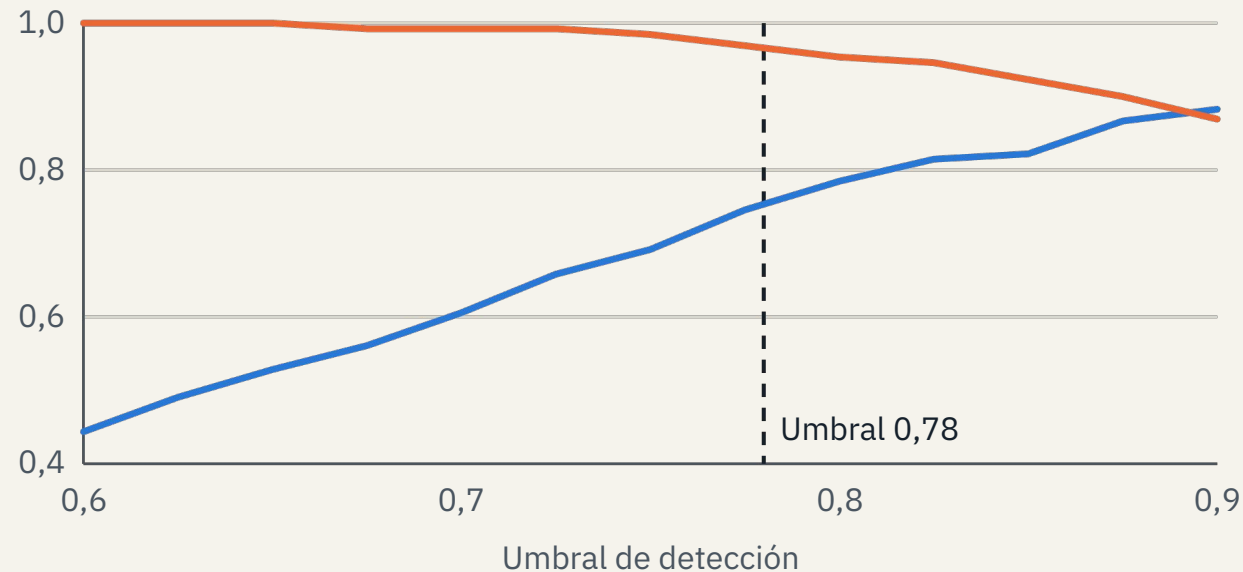


- Sin timestamp, resolución ni último login el error no cambia, y log(amount) lo empeora: **6 entradas, monto sin transformar**
- Con 262 muestras (5 % del train) el error ya es casi el mismo: el modelo tiene 7 pesos y los datos no son el límite

Umbral de detección

— Precisión — Recall

Precisión / recall



Recomendación: perceptrón logistic con 6 entradas y umbral 0,78

Umbral fijo en el test de 10 splits

Umbral	Precisión	Recall	Recall bajo 0,95
0,75	0,672	0,982	0 de 10
0,78	0,713	0,968	0 de 10
0,80	0,751	0,962	3 de 10
0,82	0,777	0,945	6 de 10
0,85	0,805	0,898	10 de 10

Cada split elige su umbral en validación: entre 0,77 y 0,84. La mediana, 0,78, es la recomendación.

EJERCICIO 2

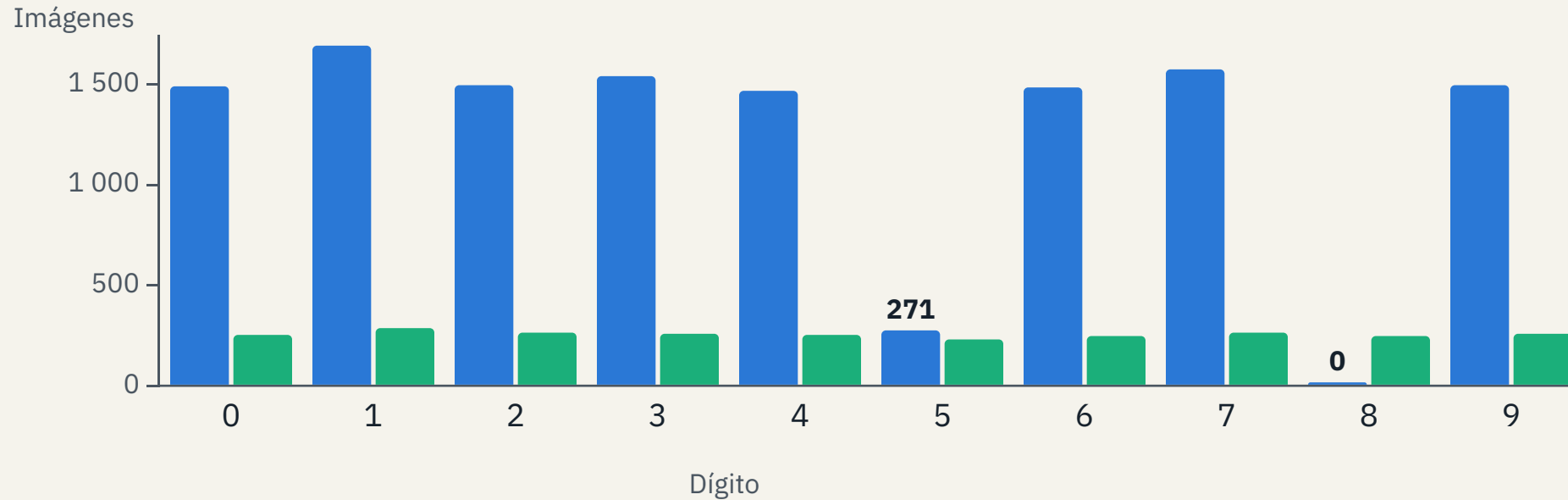
Dígitos manuscritos

Perceptrón multicapa: ¿cómo evalúo el desempeño? ¿qué variantes pruebo?

digits.csv para ajustar parámetros e hiperparámetros. digits_test.csv es el mundo real y se usa al final.

Datos de aprendizaje y de test

— digits.csv (12 449) — digits_test.csv (2 497)



- 784 entradas (un píxel cada una) y 10 salidas one-hot; se predice por la salida más alta
- digits.csv no tiene ningún 8 y solo 271 cincos. digits_test.csv está balanceado y el 8 es el 9,7 % del test: sin verlo, la accuracy máxima es $\approx 90\%$

¿Cómo evaluamos el desempeño?

Accuracy

Fracción de dígitos bien clasificados, tomando la salida más alta de las 10.
Es la métrica del enunciado, pero sola no dice qué clase falla.

Por clase

Aciertos y precisión de cada dígito y matriz de confusión: muestran qué dígito se confunde con cuál.

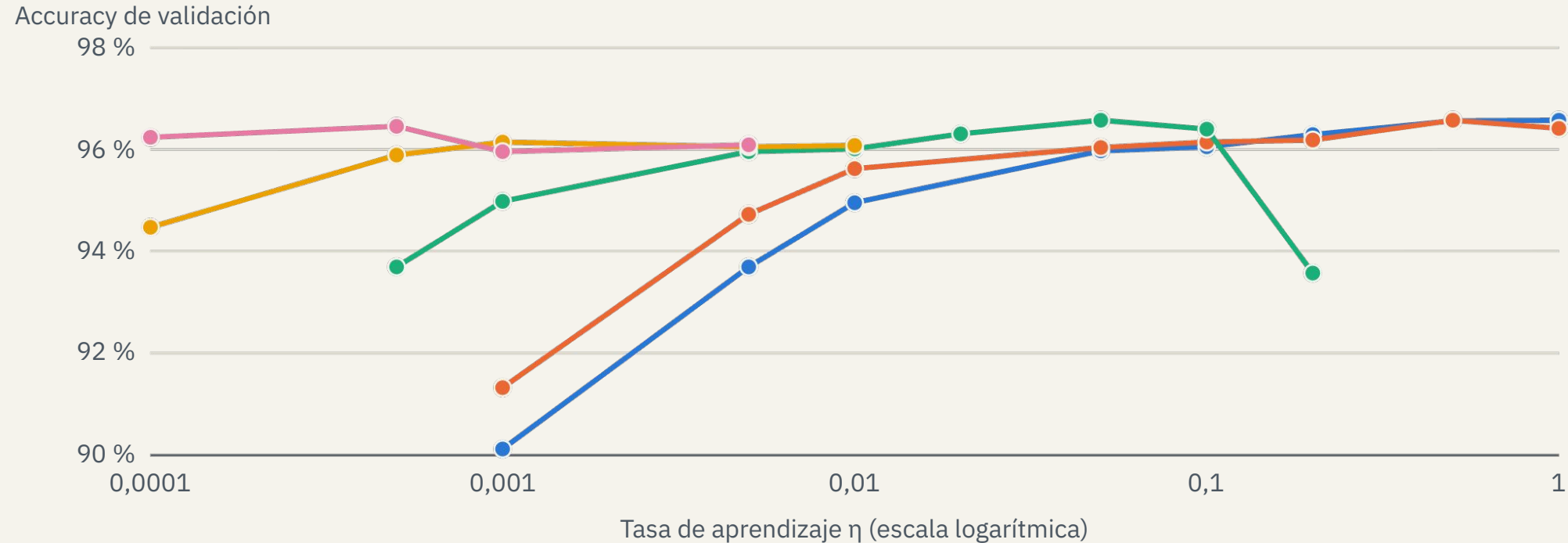
Ajuste y test

Los hiperparámetros se eligen con una validación de 20 % de digits.csv. digits_test.csv se usa una sola vez, al final.

Además: varias seeds (pesos iniciales y shuffle) y el MSE de train contra el de validación, para ver el sobreajuste.

Tasa de aprendizaje de cada optimizador

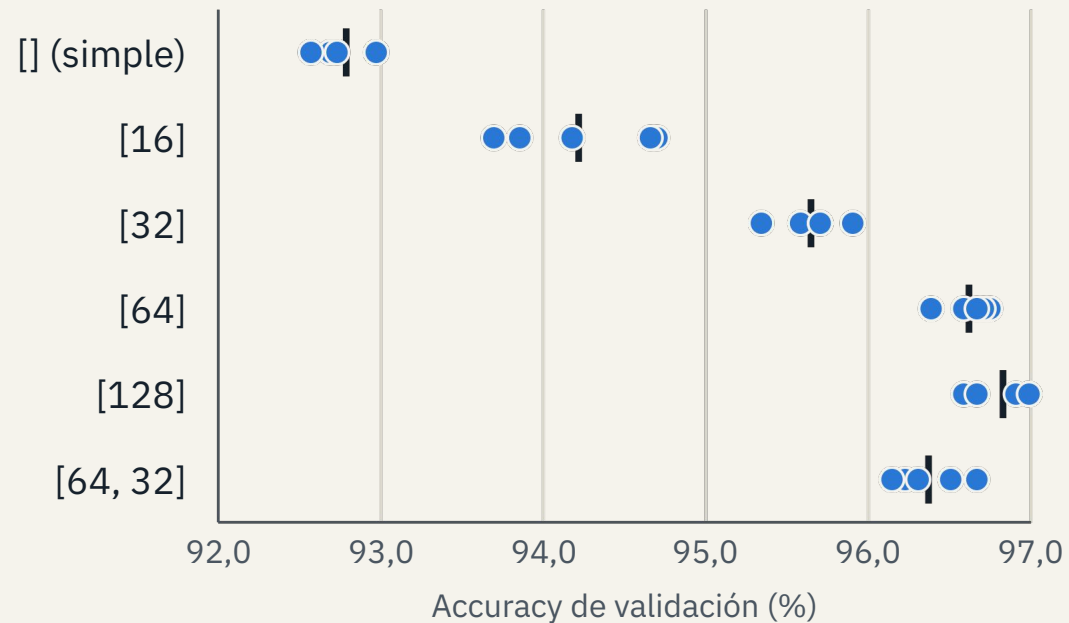
GD η adaptativo Momentum RMSProp Adam



Mejor η : GD 0,5, η adaptativo 0,5, momentum 0,05, RMSProp 0,001 y Adam 0,0005. GD con η 5 y momentum con η 0,5 divergen (33 % y 16 %).

Arquitectura de la red

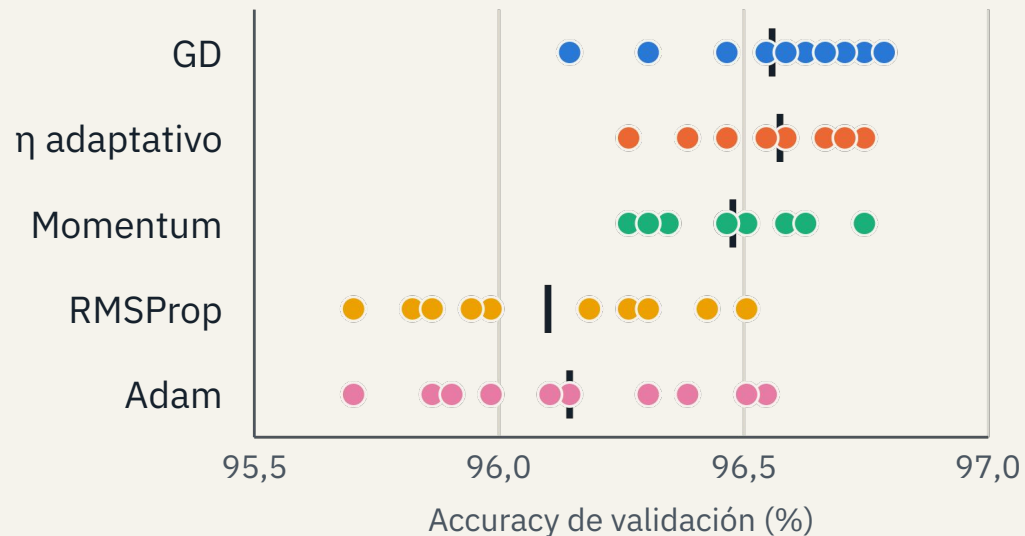
Accuracy de validación según las capas ocultas



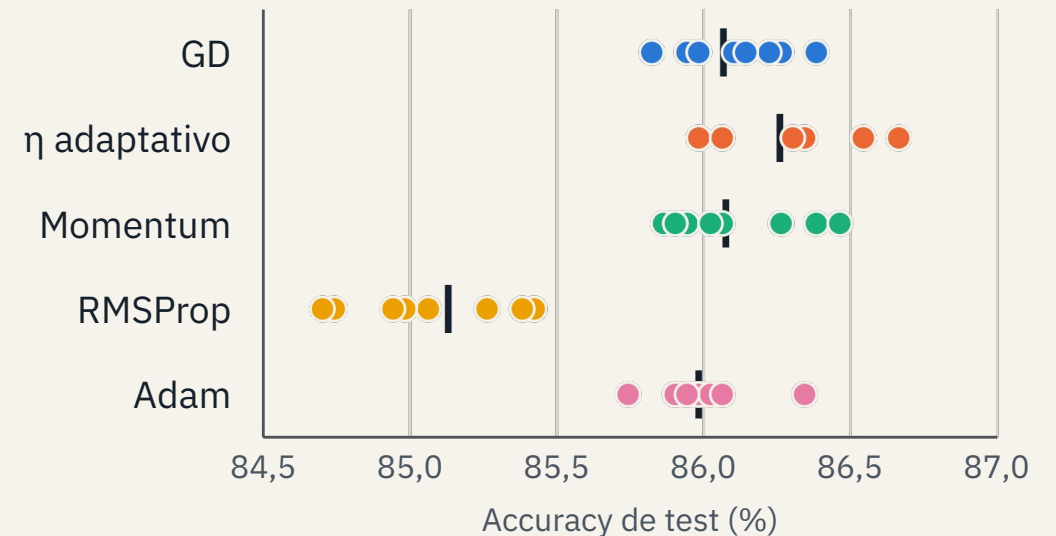
- Sin capa oculta (perceptrón simple de 10 salidas): 92,8 %. Con [64]: 96,6 %
- [128] suma 0,2 puntos y tarda el doble (41 s contra 22 s por corrida)
- Dos capas, [64, 32], no mejoran a una de 64 (96,4 %)
- **Se usa [64]**, por costo contra beneficio

Los cinco con su mejor η

Validación (20 % de digits.csv)



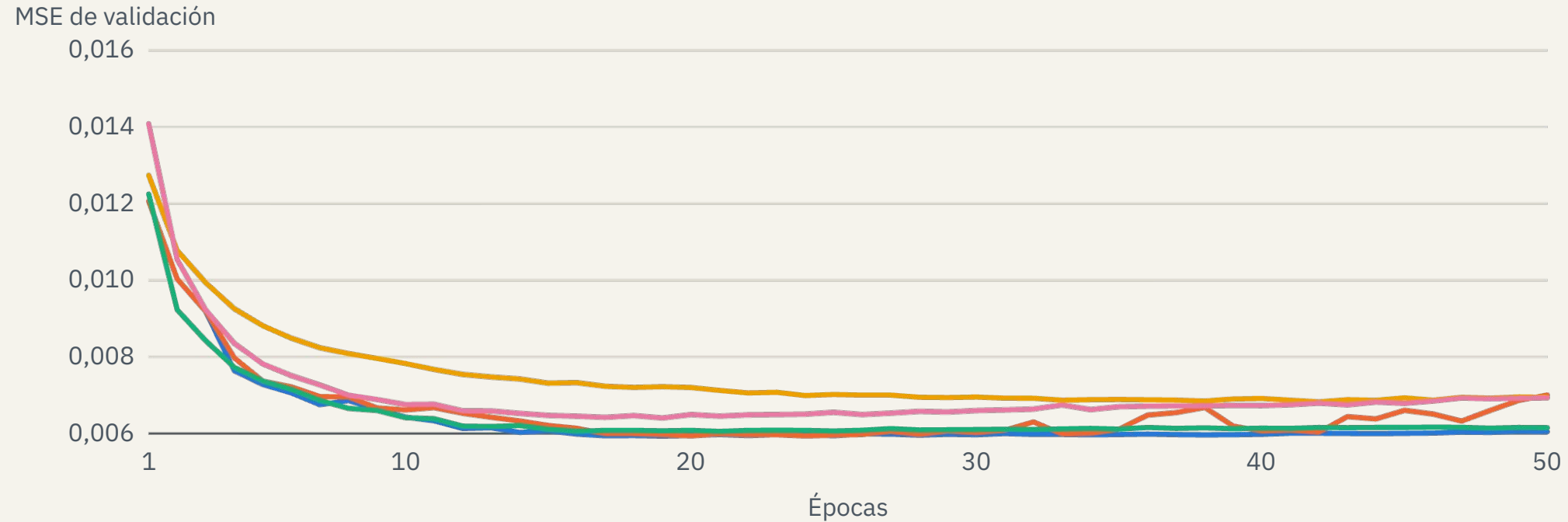
Test (digits_test.csv), entrenando solo con digits.csv



- En validación, GD, momentum y η adaptativo no se distinguen. RMSProp y Adam quedan 0,4 a 0,5 puntos por debajo del mejor, con p menor a 0,03 (corrección de Holm)
- En test, entrenando solo con digits.csv, los cinco quedan entre 85,1 y 86,3 %: el techo lo pone el 8, que no se vio

Velocidad de aprendizaje

GD η adaptativo Momentum RMSProp Adam



Todos bajan de 0,010 en 2 o 3 épocas y después casi no mejoran. RMSProp y Adam quedan más arriba, y η adaptativo sube hacia el final (0,0060 a las 25 épocas, 0,0070 a las 50). GD y η adaptativo tardan 22 o 23 s por corrida; Adam y RMSProp, 46 a 50 s.

Evaluación final en test

0	1	2	3	4	5	6	7	8	9	0
243	1	1	1	280	2	2	248	2	3	3
2	247	2	4	2	240	3	5	7	2	17
1	181	10	1	4	6	3	1	2	1	1
231	7	6	2	247	2	8	6	15	23	78
13	44	16	4	44	9	3	5	3	238	

Filas: dígito real. Columnas: dígito predicho

86,3 ± 0,2 %

accuracy en test, solo con digits.csv; media y desvío de 10 seeds (86,0 a 86,7 %)

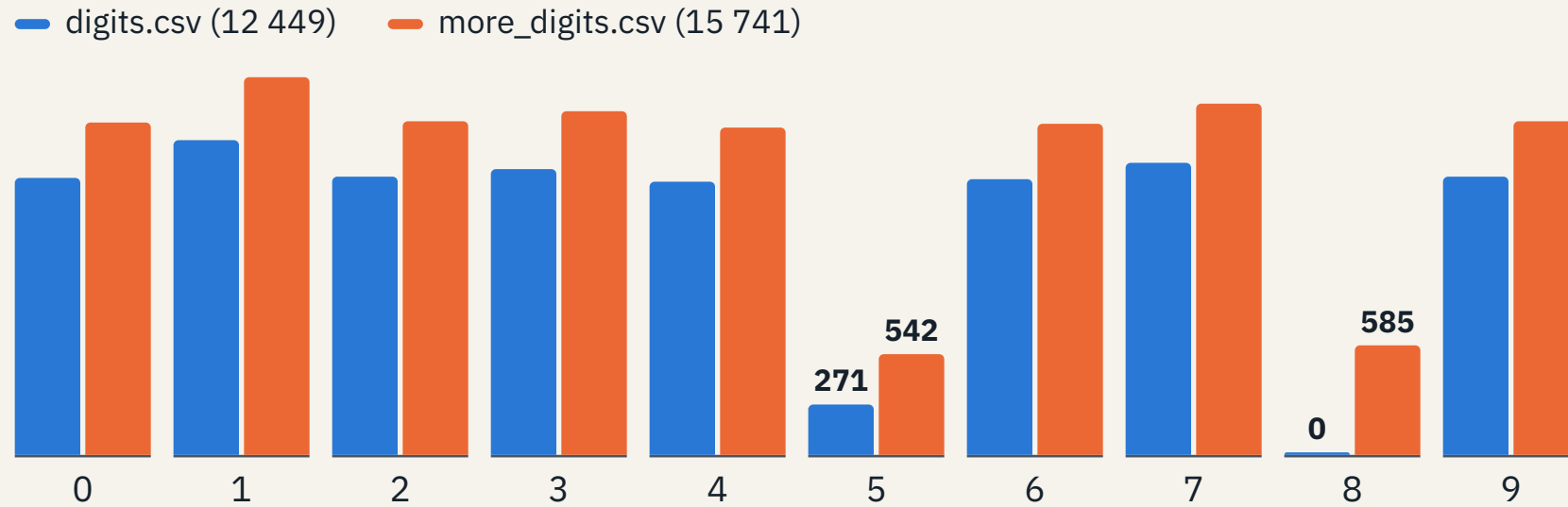
- η adaptativo (η 0,5), [64] logistic, 50 épocas: el ganador por validación
- El 8 no está en el entrenamiento: 0 % de aciertos en sus 243 imágenes, que van sobre todo a 3, 5 y 9. En los otros 9 dígitos, 95,6 %

EJERCICIO 3

Más datos, meta de 98 %

El cliente pide una accuracy de 98 % o más con el dataset nuevo. Resultado en test: 98,7 %.

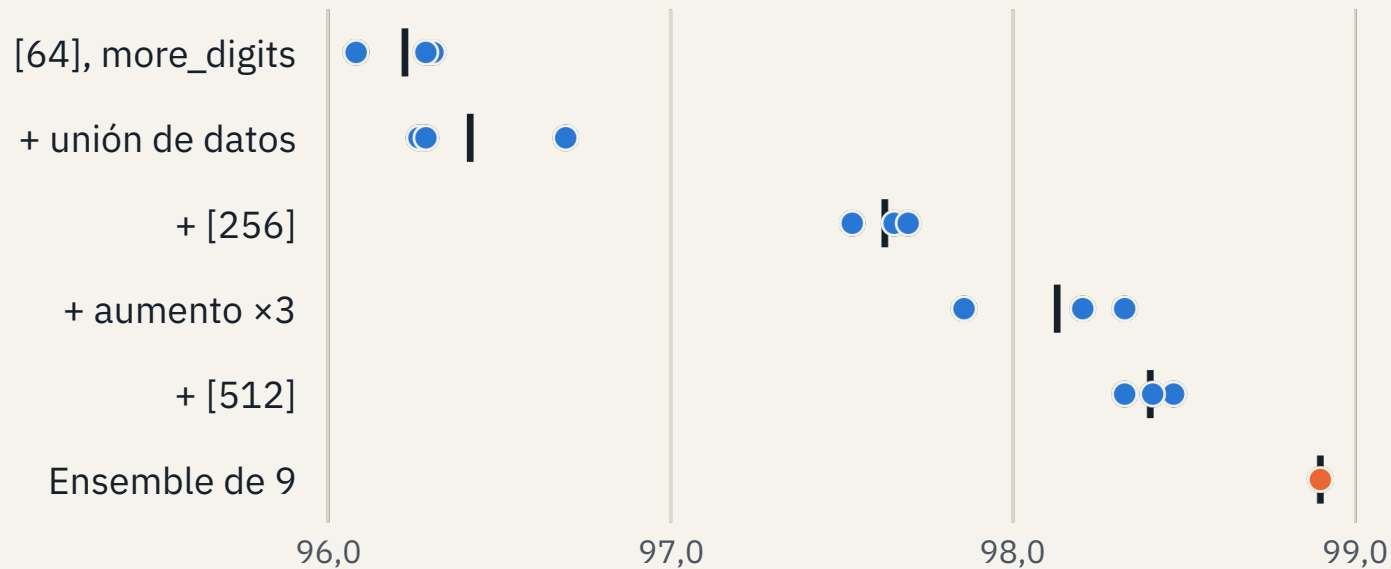
Qué cambia en los datos



- more_digits.csv sí tiene 8 (585) y duplica los cincos (271 a 542). Comparte 3689 imágenes con digits.csv: la unión sin repetidos tiene 24 501
- El test es el mismo, digits_test.csv, y no comparte ninguna imagen con ninguno de los dos: los resultados son comparables con los del Ejercicio 2

Qué aportó cada técnica

Accuracy de validación, técnica por técnica



- La capacidad y el aumento de datos suman más (1,2 y 0,5 puntos); la unión suma 0,2
- Aumento: desplazamientos de hasta 2 píxeles y rotaciones de hasta 10°, 2 copias por imagen
- Ensemble: promedio de las salidas de 9 redes, tres arquitecturas con 3 seeds cada una
- No ayudaron: centrar las imágenes (97,4 contra 97,6), 100 épocas (97,7) y otros η (97,6 y 97,2)

Mejor resultado: 98,7 % en test

0	1	2	3	4	5	6	7	8	9	0
243	1	1	1	283	2	255	2	3	250	4
244	1	5	2	1	2	216	2	6	1	1
1	1	235	7	256	8	1	1	1	1	1
1	237	9	2	2	246					

Filas: dígito real. Columnas: dígito predicho

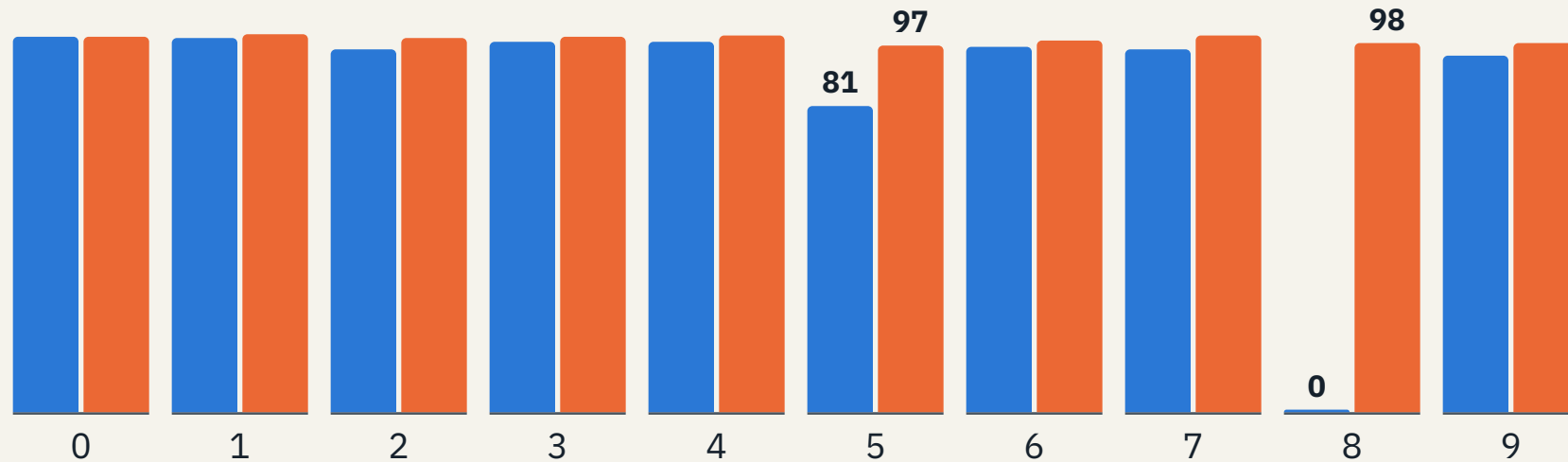
98,7 %

accuracy en test del ensemble de 9 redes. Meta del cliente: 98 %, cumplida

- 32 errores en 2497 imágenes (el 98 % admite 50), sin una confusión dominante: el máximo es 2 por par
- Una sola red [512]: $98,3 \pm 0,1$ % (3 seeds), que ya supera el 98 %
- El 5 sigue siendo la clase más débil, con 97 % de aciertos

Aciertos por clase en test

— Ejercicio 2: solo digits.csv (86,3 %) — Ejercicio 3: ensemble de 9 redes (98,7 %)



Todas las clases quedan por encima de 96,9 %. El 8 pasa de 0 a 98 % de aciertos y el 5, de 81 a 97 %.

Las tres preguntas

a

Mejor resultado

98,7 % en test, con un ensemble de 9 redes. Una sola red [512] llega a 98,3 %. Ambos superan el 98 % pedido.

b

Técnicas

Unión de digits.csv y more_digits.csv, más neuronas ([512]), aumento de datos con desplazamientos y rotaciones, y ensemble de 9 redes.

c

Otros factores

La cobertura de clases: sin 8 en el entrenamiento el techo era $\approx 90\%$. Y que el test sea del mismo origen: la validación predijo el test con 0,2 puntos de error.

Meta cumplida con margen: 32 errores en 2497 imágenes, y el 98 % admite 50.

Conclusiones

Ejercicio 1

Logistic: R^2 de 0,881 contra 0,714 del lineal. Con umbral 0,78 detecta 97 % de los fraudes con precisión de 0,71. Su salida hay que calibrarla para leerla como $P(\text{fraude})$.

Ejercicio 2

[64] y su mejor η : GD, momentum y η adaptativo se igualan (96,6 % en validación) y superan a RMSProp y Adam. En test, 86,3 %: el 8 no está en digits.csv.

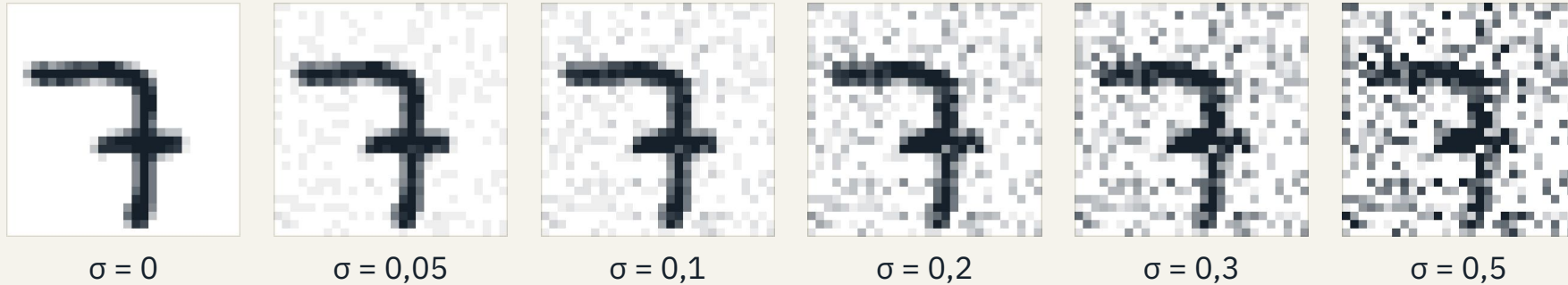
Ejercicio 3

Meta cumplida: 98,7 % en test. Los 8 explican el salto a 95,5 %; la unión de datos, [512], aumento de datos y un ensemble de 9 redes, el resto.

Opcionales

TP3 · Fraude y dígitos manuscritos

Ruido sobre las imágenes de test



Ruido

$N(0, \sigma^2)$ en cada píxel, recortado a $[0, 1]$. Seis niveles, de σ 0 a 0,5.

Repeticiones

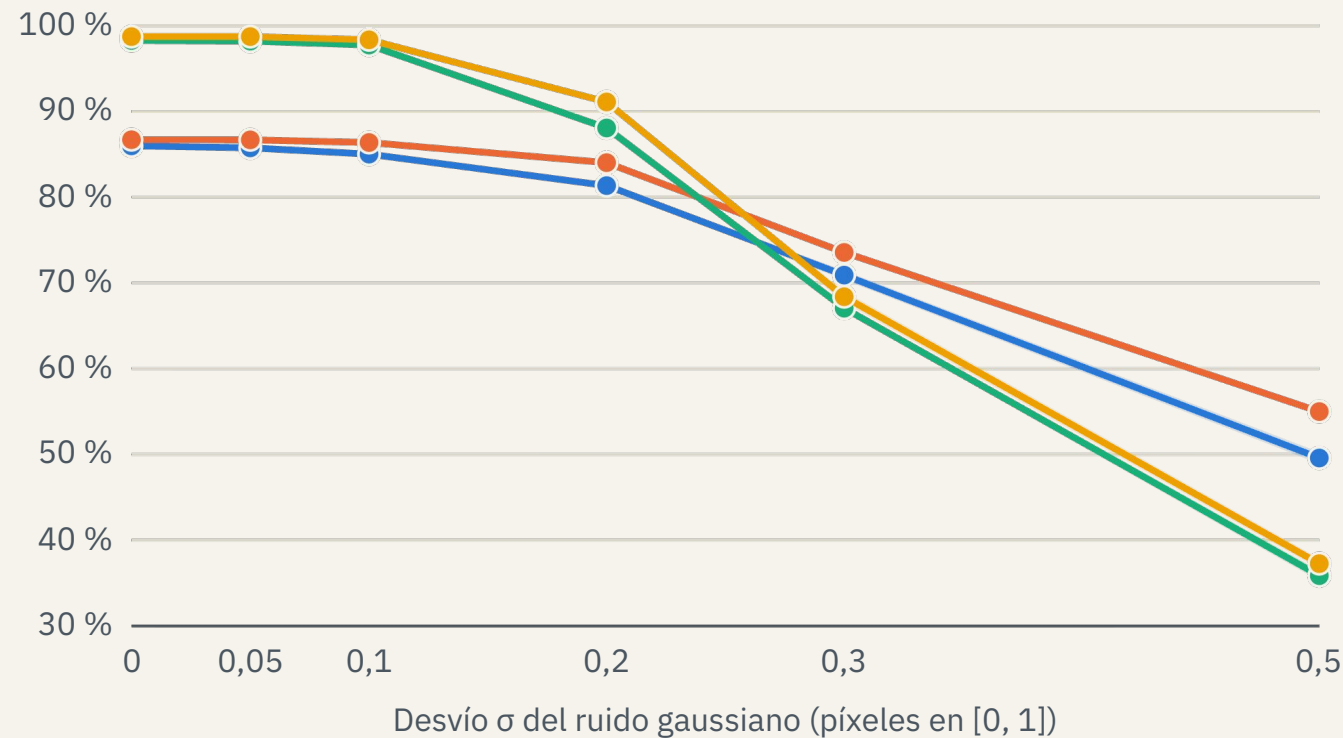
10 seeds de ruido por σ . Una seed usa el mismo ruido en todos los σ , escalado.

Modelos

Una red y el ensemble de cada ejercicio, ya entrenados: no se reentrenó nada.

Accuracy con ruido

— Ej. 2, una red — Ej. 2, ensemble de 10 — Ej. 3, una red — Ej. 3, ensemble de 9



- Hasta $\sigma 0,1$ casi no cambia: todos pierden menos de 1 punto
- **El Ej. 3 cae más rápido:** a $\sigma 0,2$ sigue arriba (91 contra 84), pero desde $\sigma 0,3$ queda debajo del Ej. 2 (68 contra 74)
- **El ensemble ayuda siempre:** en el Ej. 2, de +0,7 puntos sin ruido a +5,4 a $\sigma 0,5$
- Todas las diferencias tienen el mismo signo en las 10 seeds ($p = 0,002$)

Qué dígitos sufren el ruido

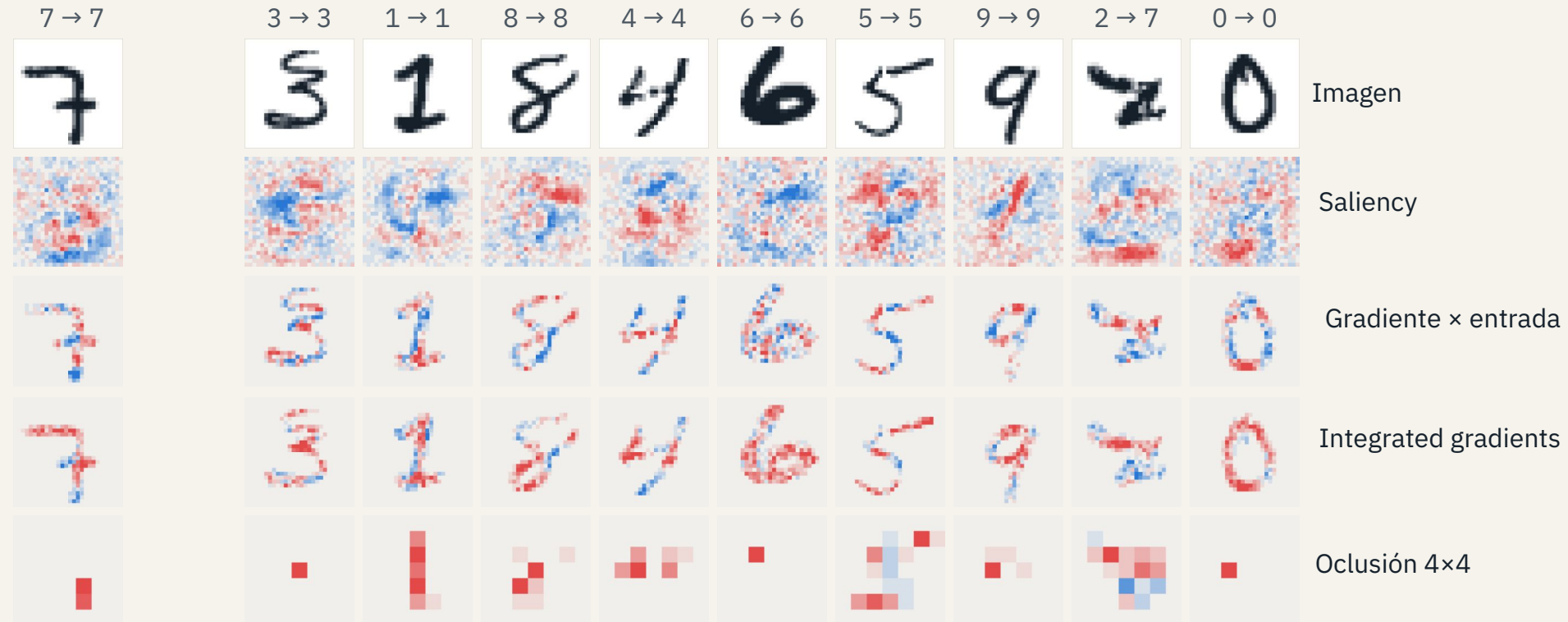
Aciertos por dígito en % con σ 0,2, media de 10 seeds de ruido

	0	1	2	3	4	5	6	7	8	9	
	99	81	96	99	91	72	95	84	0	94	Ej. 2, una red
	99	88	95	100	95	81	96	88	0	94	Ej. 2, ensemble de 10
	100	52	95	98	91	86	94	84	98	88	Ej. 3, una red
	100	71	97	99	93	90	95	85	99	86	Ej. 3, ensemble de 9

- **El 1 es el que más cae.** En el Ej. 3 baja de 100 a 71 % (una red sola, a 52 %) y la mayoría **se va a 8**: el ruido le pone tinta alrededor a un trazo fino
- El Ej. 2 no puede elegir 8 porque nunca lo vio: sus unos van a 3 y a 2, y caen menos (de 99 a 88 %)
- El 8 del Ej. 2 sigue en 0 % con cualquier σ : el techo de ≈ 90 % no cambia

Cuatro métodos de atribución

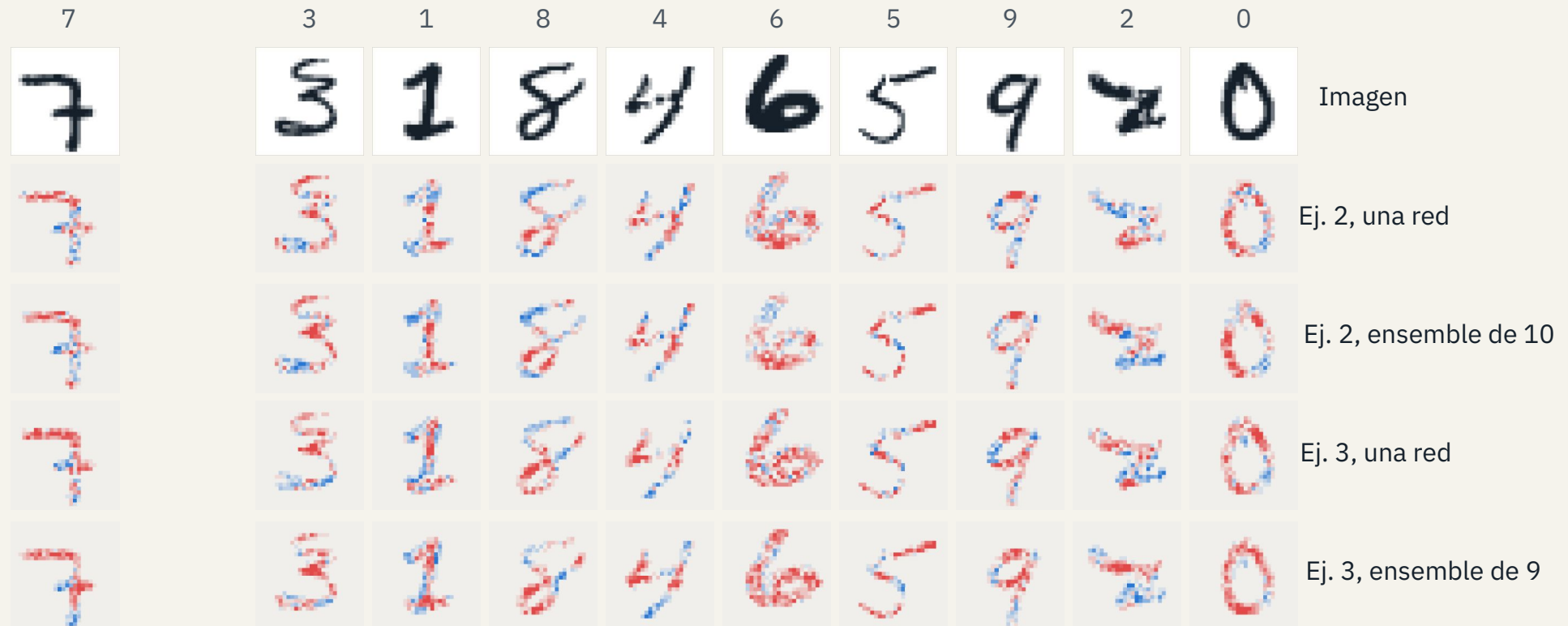
— a favor de la clase predicha — en contra



Saliency marca también el fondo; gradiente × entrada e integrated gradients siguen el trazo. La oclusión no usa gradientes: es el control.

Ejercicio 2 contra Ejercicio 3

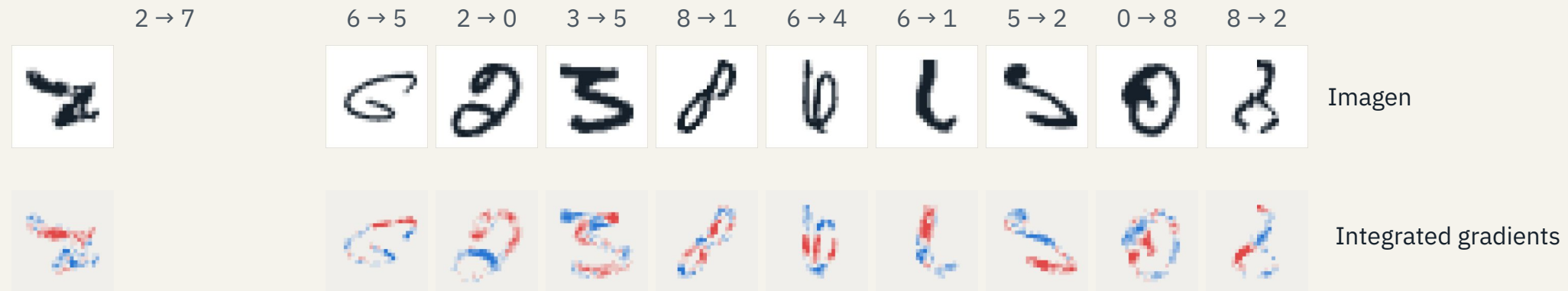
— a favor de la clase predicha — en contra



Recuadro: mal clasificada. El Ej. 2 nunca vio un 8 y lo manda a 6, apoyado en el rulo de abajo. El Ej. 3 lo acierta.

Dónde se equivoca el Ejercicio 3

— a favor de la clase predicha — en contra

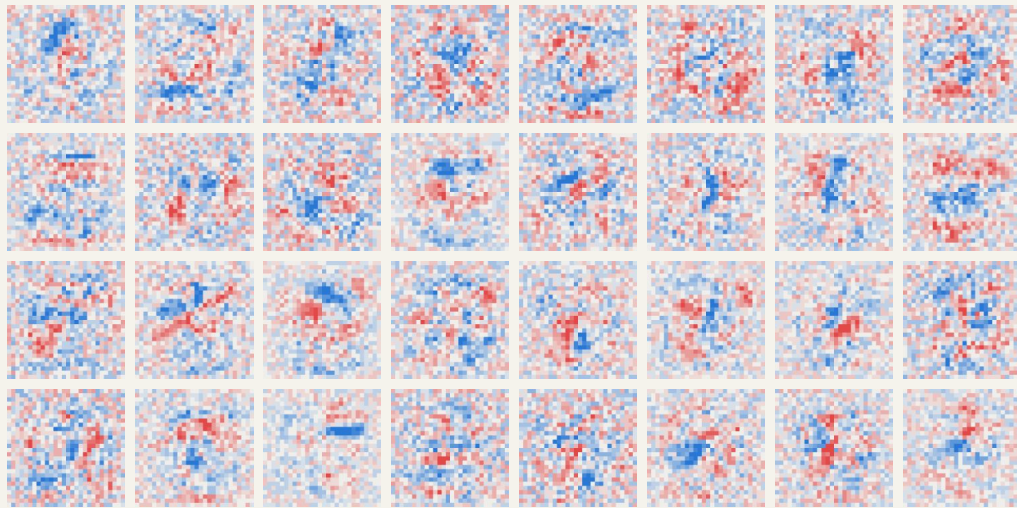


- **Son dígitos ambiguos:** un 3 de barra plana que pasa a 5, un 2 con rulo que pasa a 0, un 6 acostado que pasa a 5
- La evidencia a favor (rojo) cae en la parte que se parece a la clase elegida: la barra del 5, el rulo del 0
- No hay una confusión dominante: de los 32 errores, el máximo es 2 por par de dígitos

Lo que la atribución no explica

Pesos de entrada de 32 neuronas ocultas, Ej. 2

— peso positivo — peso negativo



Saturación

En las imágenes limpias el Ej. 3 da $O_k \approx 1$ y el gradiente es casi 0: mediana 10^{-6} , contra $2 \cdot 10^{-3}$ del Ej. 2. Un mapa local no anticipa la caída con $\sigma \geq 0,3$.

Pesos sin entrenar

Un píxel que vale 0 en todo el train deja sus pesos en el valor inicial: el borde ruidoso de los mapas. Son 97 píxeles en el Ej. 2 y 4 en el Ej. 3.

Fin.

TP3 · Fraude y dígitos manuscritos